

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
```

```
(train_images, _), (_, _) = tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5

BUFFER_SIZE = 60000
BATCH_SIZE = 256
train_dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 2s 0us/step

```
def build_generator():
    model = tf.keras.Sequential()

    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7, 7, 256)))

    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1),
                                     padding="same", use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2),
                                     padding="same", use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2),
                                     padding="same", use_bias=False,
                                     activation="tanh"))

    return model
```

```
generator = build_generator()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
def build_discriminator():
    model = tf.keras.Sequential()

    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2),
                            padding="same", input_shape=(28, 28, 1)))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2),
                            padding="same"))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation="sigmoid"))

    return model
```

```
discriminator = build_discriminator()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

cross_entropy=tf.keras.losses.BinaryCrossentropy()

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

```

```

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

```

```

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(
        gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(
        zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(
        zip(gradients_of_discriminator, discriminator.trainable_variables))

    return gen_loss, disc_loss

```

```

def train(dataset, epochs):
    for epoch in range(epochs):
        for image_batch in dataset:
            train_step(image_batch)

        print(f"Epoch {epoch+1} completed")

        if (epoch + 1) % 5 == 0:
            generate_and_save_images(generator, epoch+1)

```

```

def generate_and_save_images(model, epoch):
    noise = tf.random.normal([16, 100])
    predictions = model(noise, training=False)

    plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions[i, :, :, 0], cmap="gray")
        plt.axis("off")

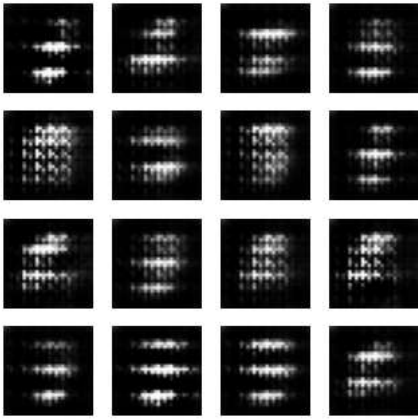
    plt.show()

```

```
train(train_dataset, epochs=2)
```

```
Epoch 1 completed
Epoch 2 completed
```

```
generate_and_save_images(generator, 2)
```



EXPIEMENT 4A

```
# =====
# 1 Install Required Libraries
# =====
!pip install --quiet transformers torch

# =====
# 2 Import Libraries
# =====
from transformers import pipeline
from IPython.display import display, Markdown

# =====
# 3 Load Models (All Local)
# =====
gpt2 = pipeline("text-generation", model="gpt2")
distilgpt2 = pipeline("text-generation", model="distilgpt2")
gpt2_medium = pipeline("text-generation", model="gpt2-medium")

# =====
# 4 Common Prompt
# =====
prompt = "Write a haiku about AI and nature."

# =====
# 5 Generate Responses
# =====
competitors = [
    "GPT2",
    "DistilGPT2",
    "GPT2-Medium"
]

answers = [
    gpt2(prompt, max_length=60, temperature=0.7)[0]["generated_text"],
    distilgpt2(prompt, max_length=60, temperature=0.7)[0]["generated_text"],
    gpt2_medium(prompt, max_length=60, temperature=0.7)[0]["generated_text"]
]

# =====
# 6 Display Outputs
# =====
display(Markdown("## 🚀 Model Outputs\n"))

for comp, ans in zip(competitors, answers):
    display(Markdown(f"### 🤖 {comp}\n{ans}\n---"))

# =====
# 7 Simple Automatic Evaluation
# =====
def score_response(text):
    score = 0
    score += len(text.split()) # length
    score += text.count("\n") * 5 # poetic structure bonus
    return score
```

```
scores = [(comp, score_response(ans)) for comp, ans in zip(competitors, answers)]
scores.sort(key=lambda x: x[1], reverse=True)

# =====
# 8 Display Ranking
# =====
display(Markdown("## 🏆 Final Ranking (Best → Worst)\n"))

for i, (model, score) in enumerate(scores):
    display(Markdown(f"***Rank {i+1}: {model}** - Score: {score}"))
```

Loading weights: 100% 148/148 [00:00<00:00, 709.33it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: gpt2

Key	Status	
h.{0...11}.attn.bias	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Loading weights: 100% 76/76 [00:00<00:00, 411.40it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: distilgpt2

Key	Status	
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Loading weights: 100% 292/292 [00:00<00:00, 698.30it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: gpt2-medium

Key	Status	
h.{0...23}.attn.bias	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Both `max\_new\_tokens` (=256) and `max\_length` (=60) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to this issue for more information.

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

Both `max\_new\_tokens` (=256) and `max\_length` (=60) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to this issue for more information.

🔥 Model Outputs

🤖 GPT2

Write a haiku about AI and nature.

"Humans have evolved since ancient times to become the most intelligent beings on the planet," said Dr. David N. Friedman of the Stanford University School of Medicine in a press release. "In the past, this has been an issue for us. Humans have been working on this problem for many decades, but now that we have made progress it's time to take a step back and realize that we are going to change things. We are not going to be able to solve this problem from a technological standpoint. So we need to find ways to communicate with each other and to solve the problem, and I am very optimistic about this project."

This is the fifth time the Stanford team has used a "mind-controlled" robot, and it was created by the University of Texas at Austin, Dr. David Friedman said.

"It's probably the most powerful robot in the world," he said. "It can act like a human and use its own emotions to communicate with other robots. It can even act like a robot, so it can communicate with itself and it can even communicate with other robots. It's a really beautiful robot, and it's probably the most powerful machine in the world. It can act like a human and use its own emotions"

🤖 DistilGPT2

Write a haiku about AI and nature.

The first part of this post is about the way AI works. I will outline how we can use AI to help us understand humans better, but then I will explain how we can use AI to help us solve the problems that we have.

```
!pip install -q transformers torch accelerate pandas pyarrow
```

```
import torch
import re
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
ds = load_dataset("openai/gsm8k", "main")
data = ds["train"]

print("Total training samples:", len(data))
print("\nSample example:\n")
print(data[0])
```

This post is a kind of summary of the main points. Please check the previous updates and updates to the post for the detailed explanation. The basic idea of LLaMA is intelligence is a question that comes up often when AI is discussed. For example, in the article "Artificial Intelligence Needs a Name" we mentioned that we're interested in a intelligent system that is able to learn from the way humans use words, and not from anything else. You will be able to reuse this secret in all of your notebooks. There are various kinds of AI, so one can think of them as: Please note that authentication is recommended but still optional to access public models or datasets. A system that is able to learn from the way humans use words, and not from anything else.

WARNING: This is a system that is able to learn from the way humans use words, but doesn't use them.

WARNING: This is a system that is able to learn from the way humans use words, but doesn't use them. Please set a HF_TOKEN to enable higher rate limits and faster d WARNING: This is a system that is able to learn from the way humans use words, but doesn't use them. Please set a HF_TOKEN to enable higher rate limits and faster d WARNING: This is a system that is able to learn from the way humans use words, but doesn't use them. Please set a HF_TOKEN to enable higher rate limits and faster d

Examples of such system

Generating train split: 100%	7473/7473 [00:00<00:00, 110557.85 examples/s]
Generating test split: 100%	1319/1319 [00:00<00:00, 60316.47 examples/s]
Rank 1: DistilGPT2 — Score: 30.1	
Rank 2: GPT2-Medium — Score: 30.1	

Sample example:

{'question': 'Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

Using device: cuda

```
model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    low_cpu_mem_usage=True
).to(device)

model.eval()

print("Model loaded successfully!")
```

config.json: 100%	608/608 [00:00<00:00, 61.0kB/s]
tokenizer_config.json: 1.29k/? [00:00<00:00, 31.3kB/s]	
tokenizer.json: 1.84M/? [00:00<00:00, 21.8MB/s]	
tokenizer.model: 100%	500k/500k [00:00<00:00, 1.03MB/s]
special_tokens_map.json: 100%	551/551 [00:00<00:00, 16.3kB/s]
`torch_dtype` is deprecated! Use `dtype` instead!	
model.safetensors: 100%	2.20G/2.20G [00:28<00:00, 197MB/s]
Loading weights: 100%	201/201 [00:06<00:00, 28.89it/s, Materializing param=model.norm.weight]
generation_config.json: 100%	124/124 [00:00<00:00, 14.3kB/s]
Model loaded successfully!	

```
def build_prompt(question, mode="zero_shot", exemplars=None):

    if mode == "zero_shot":
        return f""Solve the following math problem.
        Give only the final numeric answer.

    Question: {question}
    Answer: ""

    elif mode == "few_shot":
```

```

prompt = "Here are some solved examples:\n\n"
for q, a in exemplars:
    prompt += f"Q: {q}\nA: {a}\n\n"
prompt += f"Now solve this problem.\nQ: {question}\nA:"
return prompt

elif mode == "cot":
    return f""Solve the following math problem step by step.
After reasoning, give the final answer.

Question: {question}
Let's think step by step.""

```

```

def generate_answer(prompt, max_new_tokens=128):

    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

```

```

def extract_last_number(text):
    numbers = re.findall(r"-?\d+\.?\d*", text)
    return numbers[-1] if numbers else None

```

```

def get_few_shot_examples(k=3):
    examples = []
    for i in range(k):
        q = data[i]["question"]
        a = data[i]["answer"].split("####")[-1].strip()
        examples.append((q, a))
    return examples

few_shot_examples = get_few_shot_examples(3)

```

```

def evaluate_mode(mode, n_samples=20):

    correct = 0
    hallucinations = 0

    for i in range(n_samples):

        question = data[i]["question"]
        gt_answer = data[i]["answer"].split("####")[-1].strip()

        if mode == "few_shot":
            prompt = build_prompt(question, mode, few_shot_examples)
        else:
            prompt = build_prompt(question, mode)

        output = generate_answer(prompt)

        pred = extract_last_number(output)
        gt = extract_last_number(gt_answer)

        if pred is None:
            hallucinations += 1
        elif pred == gt:
            correct += 1

        print(f"[{mode}] Done {i+1}/{n_samples}")

    accuracy = correct / n_samples
    hallucination_rate = hallucinations / n_samples

    print("\n-----")
    print("Mode:", mode)
    print("Accuracy:", round(accuracy,3))
    print("Hallucination Rate:", round(hallucination_rate,3))

```

```
print("-----\n")
```

```
return accuracy, hallucination_rate
```

```
acc_zero, hall_zero = evaluate_mode("zero_shot", n_samples=100)
acc_few, hall_few = evaluate_mode("few_shot", n_samples=100)
acc_cot, hall_cot = evaluate_mode("cot", n_samples=100)
```

```
[zero_shot] Done 1/100
[zero_shot] Done 2/100
[zero_shot] Done 3/100
[zero_shot] Done 4/100
[zero_shot] Done 5/100
[zero_shot] Done 6/100
[zero_shot] Done 7/100
[zero_shot] Done 8/100
[zero_shot] Done 9/100
[zero_shot] Done 10/100
[zero_shot] Done 11/100
[zero_shot] Done 12/100
[zero_shot] Done 13/100
[zero_shot] Done 14/100
[zero_shot] Done 15/100
[zero_shot] Done 16/100
[zero_shot] Done 17/100
[zero_shot] Done 18/100
[zero_shot] Done 19/100
[zero_shot] Done 20/100
[zero_shot] Done 21/100
[zero_shot] Done 22/100
[zero_shot] Done 23/100
[zero_shot] Done 24/100
[zero_shot] Done 25/100
[zero_shot] Done 26/100
[zero_shot] Done 27/100
[zero_shot] Done 28/100
[zero_shot] Done 29/100
[zero_shot] Done 30/100
[zero_shot] Done 31/100
[zero_shot] Done 32/100
[zero_shot] Done 33/100
[zero_shot] Done 34/100
[zero_shot] Done 35/100
[zero_shot] Done 36/100
[zero_shot] Done 37/100
[zero_shot] Done 38/100
[zero_shot] Done 39/100
[zero_shot] Done 40/100
[zero_shot] Done 41/100
[zero_shot] Done 42/100
[zero_shot] Done 43/100
[zero_shot] Done 44/100
[zero_shot] Done 45/100
[zero_shot] Done 46/100
[zero_shot] Done 47/100
[zero_shot] Done 48/100
[zero_shot] Done 49/100
[zero_shot] Done 50/100
[zero_shot] Done 51/100
[zero_shot] Done 52/100
[zero_shot] Done 53/100
[zero_shot] Done 54/100
[zero_shot] Done 55/100
[zero_shot] Done 56/100
[zero_shot] Done 57/100
[zero_shot] Done 58/100
```

Start coding or [generate](#) with AI.

